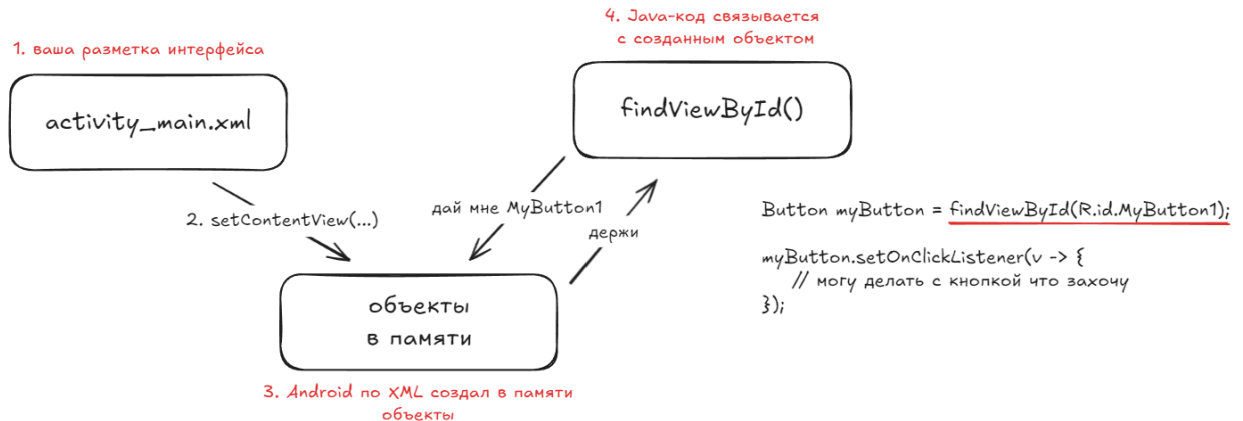


ЛАБОРАТОРНАЯ РАБОТА №5

ИСПОЛЬЗОВАНИЕ АДАПТИВНЫХ ЗНАЧЕНИЙ И КНОПОК-ПЕРЕКЛЮЧАТЕЛЕЙ

Как разметка связана с кодом?

XML – это чертёж, а не сам экран. Каждый тег **<Button>**, **<TextView>** – это описание будущего объекта, не более.



При вызове **setContentView()** система читает XML и создаёт java-объекты – настоящие **Button** и **TextView** в памяти, выстроенные по разметке.

Каждому элементу с атрибутом **android:id** присваивается номер-константа. Все константы лежат в автоматически генерируемом классе R (resource - ресурсы), например **R.id.btnOrder**.

findViewById(R.id.btnOrder) ищет среди созданных объектов элемент с этим номером и возвращает его.

Часть 1. Разметка экрана

Полный файл **activity_main.xml** скопируйте его (полностью набирать XML руками не будем, в этом нет смысла):

```
<?xml version="1.0" encoding="utf-8"?>  
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:orientation="vertical"  
    android:id="@+id/main"  
    android:padding="16dp">  
  
    <TextView  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:text="Закажи напиток"  
        android:textSize="24sp"  
        android:textStyle="bold" />  
  
    <TextView  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:layout_marginTop="16dp"  
        android:text="Размер напитка"  
        android:textSize="16sp" />  
  
    <RadioGroup  
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content">

        <RadioButton
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="250 мл - 100 ₺" />

        <RadioButton
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:checked="true"
            android:text="350 мл - 150 ₺" />

        <RadioButton
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="450 мл - 200 ₺" />
    </RadioGroup>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Добавки"
        android:textSize="16sp" />

    <CheckBox
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Сироп (+30 ₺)" />

    <CheckBox
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Растительное молоко (+40 ₺)" />

    <ToggleButton
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:textOff="Самовывоз"
        android:textOn="Доставка" />

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Адрес доставки"
        android:inputType="textPostalAddress"
        android:visibility="gone" />

    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Оформить заказ" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:textSize="28sp"
        android:textStyle="bold"
        android:text="Тут будет цена"/>
</LinearLayout>
```

Разберем из разметки четыре вещи:

1. **RadioGroup** объединяет три **RadioButton** в группу: выбор одного автоматически снимает остальные (кофе же не может быть двух размеров сразу). Средний размер предвыбран, у него есть:

```
android:checked="true"
```

2. **CheckBox** же стоят сами по себе: сироп и молоко не исключают друг друга.
3. **ToggleButton** имеет два устойчивых состояния и подписи для каждого:

```
android:textOff="Самовывоз"  
android:textOn="Доставка"
```

4. А под ним **EditText** для заполнения адреса, спрятанное атрибутом:

```
android:visibility="gone"
```

Мы покажем поле ввода из кода, когда **доставка** будет включена.

А теперь:

1. вынесите все строки из элементов в ресурсы **strings.xml**.
2. назначьте каждому элементу свой **id**, например:

```
android:id="@+id/btnOrder"
```

Часть 2. Оживляем кнопку: слушатели

Сначала создаем поля класса для элементов, с которыми будем работать, и сразу найдем их на экране:

```
public class MainActivity extends AppCompatActivity {

    private RadioGroup rgSize; 1 usage
    private CheckBox cbSyrup, cbMilk; 1 usage
    private ToggleButton tbDelivery; 1 usage
    private EditText etAddress; 1 usage
    private TextView tvPrice; 1 usage

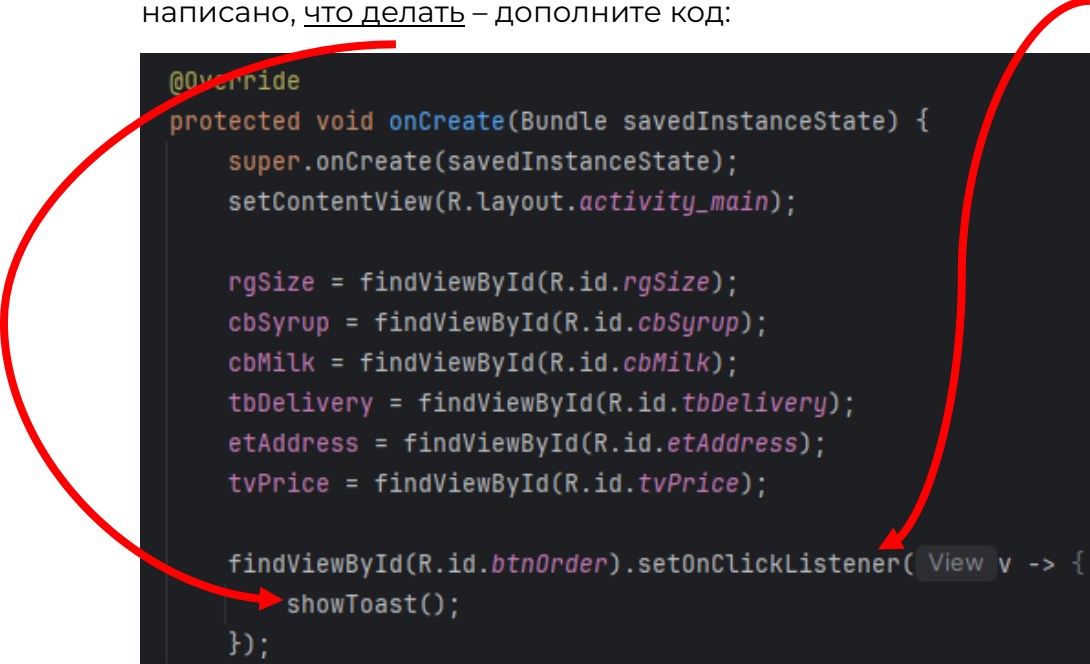
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        rgSize = findViewById(R.id.rgSize);
        cbSyrup = findViewById(R.id.cbSyrup);
        cbMilk = findViewById(R.id.cbMilk);
        tbDelivery = findViewById(R.id.tbDelivery);
        etAddress = findViewById(R.id.etAddress);
        tvPrice = findViewById(R.id.tvPrice);
    }
}
```

id у вас будут свои –
то есть те которые
вы указали ранее в
XML

Импорты Android Studio добавит сама: поставьте курсор на подчёркнутое имя и нажмите **Alt+Enter**.

Когда пользователь нажимает кнопку, система создаёт **событие**. Чтобы о нём узнать, нужно **подписаться** – передать кнопке **объект-слушатель**, в котором написано, что делать – дополните код:



```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    rgSize = findViewById(R.id.rgSize);
    cbSyrup = findViewById(R.id.cbSyrup);
    cbMilk = findViewById(R.id.cbMilk);
    tbDelivery = findViewById(R.id.tbDelivery);
    etAddress = findViewById(R.id.etAddress);
    tvPrice = findViewById(R.id.tvPrice);

    findViewById(R.id.btnOrder).setOnClickListener( View v -> {
        showToast();
    });
}

private void showToast() { 1 usage
    Log.d( tag: "Coffee", msg: "Кнопка нажата!");
    Toast.makeText( context: this, text: "Заказ принят", Toast.LENGTH_SHORT).show();
}
```

Запустите, нажмите кнопку: внизу экрана всплывает плашка, а в логах появляется ваша строка.

Откройте вкладку **Logcat** внизу и введите в фильтр Coffee – так вы найдете свои сообщения среди тысяч чужих.

Часть 3. Переключатели и живая цена

У кнопки событие – “нажатие” (или же “клик”). У переключателей – изменение состояния.

Напишем один метод **calculateTotal()**, который читает всё состояние и обновляет цену, и будем вызывать его из каждого слушателя - не “галочка прибавила 30”, а “состояние изменилось — пересчитай всё заново”. Так цена никогда не накопит ошибку, да и выглядит лаконичнее.

Цены объявите константами в начале класса:

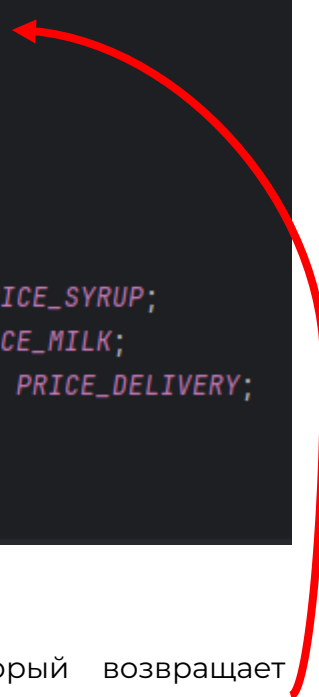
```
private static final int PRICE_SMALL = 100; 1 usage
private static final int PRICE_MEDIUM = 150; 1 usage
private static final int PRICE_LARGE = 200; 1 usage
private static final int PRICE_SYRUP = 30; 1 usage
private static final int PRICE_MILK = 40; 1 usage
private static final int PRICE_DELIVERY = 100; 1 usage
```

Напишем метод расчета цены:

```
private void calculateTotal() { 5 usages
    int price;
    int sizeId = rgSize.getCheckedRadioButtonId();
    if (sizeId == R.id.rbSmall) {
        price = PRICE_SMALL;
    } else if (sizeId == R.id.rbLarge) {
        price = PRICE_LARGE;
    } else {
        price = PRICE_MEDIUM;
    }

    if (cbSyrup.isChecked()) price += PRICE_SYRUP;
    if (cbMilk.isChecked()) price += PRICE_MILK;
    if (tbDelivery.isChecked()) price += PRICE_DELIVERY;

    tvPrice.setText("Цена = " + price);
}
```



Обратите внимание на это в **calculateTotal()**:

1. **getCheckedRadioButtonId()** – метод, который возвращает id того RadioButton, который выбран – что мы собственно и **проверяем**.
2. **isChecked()** – метод, который возвращает true/false в зависимости от того, стоит галочка или нет

Добавьте в **onCreate** подписки на события:

```
// Слушатель вешается на ГРУППУ, а не на каждую RadioButton
rgSize.setOnCheckedChangeListener(( RadioGroup group, int checkedId) -> {
    calculateTotal();
});
// Вешаем слушателей на CheckBox'ы
cbSyrup.setOnCheckedChangeListener(( CompoundButton button, boolean isChecked) -> {
    calculateTotal();
});
cbMilk.setOnCheckedChangeListener(( CompoundButton button, boolean isChecked) -> {
    calculateTotal();
});
// Вешаем слушателя на ToggleButton
tbDelivery.setOnCheckedChangeListener(( CompoundButton button, boolean isChecked) -> {
    if (isChecked) {
        etAddress.setVisibility(View.VISIBLE);    // показываем поле адреса
    } else {
        etAddress.setVisibility(View.GONE);       // прячем его и не показываем
    }
    calculateTotal();
});

// самый первый расчёт сразу при запуске экрана
calculateTotal();
```

А теперь сделайте чтобы **showToast()** – показывал итоговую цену через плашку (**Toast**), а также потом логировал.

Запустите: цена меняется, адрес появляется и исчезает, заказ принимается.

Требования к итоговой версии

1. Цена пересчитывается при изменении любого элемента, включая первый запуск.
2. Поле адреса появляется только при включённой доставке и убирает своё место.
3. Все строки – в ресурсах; текст в **sp**, отступы в **dp**.
4. Нажатие кнопки даёт всплывающее подтверждение (Toast) и запись в Logcat.

Дополнительно (по желанию)

1. Добавьте **третью добавку** “Корица +20 Р”
2. **Добавьте кнопку “Сбросить”**, возвращающую элементы в исходное состояние из кода:

```
rgSize.check(R.id.rbMedium);  
cbSyrup.setChecked(false);  
etAddress.setText("");  
etAddress.setVisibility(View.GONE);
```